

Reviewer Pack — Plethysm Coefficient Gap in Symmetric Powers of Permanent vs...

Entry f3dc4c5f9c43 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 00:43:01 UTC

Plethysm Coefficient Gap in Symmetric Powers of Permanent vs Determinant

Entry ID: f3dc4c5f9c43 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 00:43:01 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups (Plethysm Coefficients) **Field B** (complexity object): Boolean Circuit Size for Permanent Polynomial

Statement:

For any CNF formula Φ with n variables, the plethysm coefficient of the permanent polynomial ($\lambda \vdash n$) exceeds that of the determinant polynomial by a factor of $\Omega(2^{\lfloor n/2 \rfloor})$, where λ is the Young diagram corresponding to Φ 's clause structure.

Rationale (proposer's reasoning):

Plethysm coefficients encode decomposition complexity of symmetric tensors; the permanent's higher coefficients suggest intrinsic structural complexity that resists efficient circuit simulation, aligning with GCT's orbit closure separation goals.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 60df0fca3bd3b216

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations_with_replacement, permutations

def partitions(k, n):
    if k == 0:
        yield []
    elif n == 0:
        yield [0] * k
    else:
        for p in partitions(k-1, n):
            yield [p[0]+1] + p[1:]
        for p in partitions(k, n-1):
            yield [p[0]] + p[1:]

def hook_length_formulas(n, k):
    if n == 0 and k == 0:
        return 1
    elif n == 0 or k == 0:
        return 0
    else:
        hook = (n+k-1) * (k+n-k) // 2
        return hook_length_formulas(n-1, k-1) + hook_length_formulas(n-1, k)

def plethysm_coefficient(n, k):
    perm = hook_length_formulas(n, k)
    det = hook_length_formulas(n, n-k)
    return perm / det

def generate_3sat_formula(n, m):
    variables = list(range(1, n+1))
    clauses = []
    for _ in range(m):
        clause = random.sample(variables, 3)
        clauses.append(clause)
    return clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
```

```

total_metric_value = 0
instances_tested = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    m = random.randint(2*n, 3*n)
    formula = generate_3sat_formula(n, m)
    lambda_diag = [len(list(g)) for _, g in itertools.groupby(sorted(formula), key=lambda x: 1

    perm_coeff = plethysm_coefficient(n, n)
    det_coeff = plethysm_coefficient(n, 0)

    if perm_coeff <= det_coeff:
        conjecture_holds = False
        counterexample = f"n={n}, m={m}, lambda_diag={lambda_diag}"
        break

    total_metric_value += perm_coeff / det_coeff
    instances_tested += 1

return {
    "metric_name": "plethysm_coefficient_gap",
    "metric_value": total_metric_value / instances_tested,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i - 1 for i in range(5, 35)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bd90514d.py", line 88, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bd90514d.py", line 61, in run_trial
    lambda_diag = [len(list(g)) for _, g in itertools.groupby(sorted(formula), key=lambda x: len(x))]
                   ^^^^^^^^^
NameError: name 'itertools' is not defined. Did you forget to import 'itertools'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to missing itertools import, preventing data collection | next: Fix itertools import in test code and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112710
2	propose	ollama_remote	qwen3:8b	0	0	38238
3	propose	ollama_remote	qwen3:8b	0	0	80315
4	preregistration	ollama_remote	qwen3:8b	0	0	24071
5	novelty	ollama_remote	qwen3:8b	0	0	20867
6	novelty	ollama_remote	qwen3:8b	0	0	15590
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12005
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10074
9	judge	ollama_remote	qwen3:8b	0	0	17898

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 331768 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/f3dc4c5f9c43.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f3dc4c5f9c43.jsonl for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f3dc4c5f9c43.tar.gz (if generated)